

TAREA 1 - ELO320 ESTRUCTURA DE DATOS Y ALGORITMOS
PROF.: ÁLVARO COFRÉ P.
AYUDANTES: CRISTOPHER MARTÍNEZ - BYRON AGURTO
CAMPUS SAN JOAQUÍN, UTFSM. 16 DE AGOSTO DE 2024

1 EDA TECH INTERVIEWS

EDA company, una de las más prestigiosas y rentables compañías tecnológicas en el mundo liderada por Esteban Trabajos y Jeff Abrazos, está contratando nuevos programadores en C. Dado que ingresar a la compañía es igual o más difícil que entrar a Harvard, Google, Facebook o la NASA, se ha diseñado un método de selección de nuevos talentos basado en entrevistas técnicas con ejercicios de programación.

Los trabajadores que ingresan a EDA company tienen muchos beneficios, como por ejemplo: sueldos elevados, snacks ilimitados, almuerzos, gimnasio, masajes, vacaciones ilimitadas, entre otros. Por ese motivo, la cantidad de postulantes asciende a cientos de miles. Como estas posiciones son tan bien cotizadas, han surgido leaks de preguntas para ingresar a tan prestigiosa compañía e incluso han surgido productos y servicios para facilitar este proceso de entrevistas.

Ya que usted ha estudiado los conceptos de programación de C y ha sido un alumno/a destacado/a en el ramo de *Estructura de Datos y Algoritmos*, se da cuenta que con su experiencia podría ser un buen candidato para formar parte de EDA company. No obstante, usted sabe que: *todo lo bueno y que vale la pena, no viene por suerte, sino por esfuerzo, sacrificio y horas de dedicación*. Por esa razón, se propone realizar ejercicios de resolución de problemas similares a las entrevistas para prepararse y así entrar a tan prestigiosa empresa.

Con la convicción de su triunfo, solicita al profesor y los ayudantes del ramo *Estructura de Datos y Algoritmos* que le seleccione algunos ejercicios para practicar y tener una mayor probabilidad de éxito.

El profesor y los ayudantes del ramo han seleccionado 5 ejercicios, donde usted deberá resolver por separado a lo menos 3 siguiendo las reglas de entrega y consideraciones generales.

2 Ejercicio 1: Suma de enteros realmente grandes

Dado dos enteros no negativos, `num1` y `num2` que están representados por un **string**, retorne la suma de `num1` y `num2` como un **string**.

Ejemplo 1:

```
Input: num1 = "987", num2 = "1"
Output: "988"
```

Ejemplo 2:

```
Input: num1 = "999999999999999999", num2 = "1"
Output: "1000000000000000000"
```

Ejemplo 3:

```
Input: num1 = "0", num2 = "0"
Output: "0"
```

Usted debe resolver el problema considerando:

- No debe utilizar ninguna librería para manejar enteros muy grandes.
- No debe castear los número de manera directa.
- Debe considerar que los números pueden ser increíblemente grandes (más grandes que los soportados por C).
- `1 <= num1.length, num2.length <= 105`
- `num1` y `num2` son solo dígitos (no es necesario verificar si hay un caracter).

Utilice este prototipo de función para su solución:

```
char* sumaStrings(char* num1, char* num2) {
}
```

3 Ejercicio 2: Encuentre el mayor palíndromo en el texto

Dado un string `s` retorne el *mayor palíndromo* contenido en el texto

Ejemplo 1:

```
Input: s = "tangananica"
Output: "ana"
Explicación: "ana" es el mayor palíndromo en el texto
```

Ejemplo 2:

```
Input: s = "cbasdsdsdsaaqqdvvvbdaggg"
Output: "asdsdsdsa"
```

Ejemplo 3:

```
Input: s="l"
Output: "l"
```

Usted debe resolver el problema considerando:

- `1 <= s.length <= 1000`
- `s` solo puede contener dígitos y caracteres no especiales (es decir no tiene la ñ, ç, á, etc).

Utilice este prototipo de función para su solución:

```
char* elPalindromoMasLargo(char* s) {
}
```

4 Ejercicio 3: Agrupando anagramas

Dado un arreglo de string `strs`, encuentre y retorne el grupo de anagramas juntos (puede retornar la respuesta en cualquier orden)

Obs: Un **anagrama** es una palabra o frase formada al reorganizar las letras de una palabra o frase diferente, utilizando típicamente todas las letras originales exactamente una vez.

Ejemplo 1:

```
Input: strs = ["eat","tea","tan","ate","nat","bat"]
Output: [[ "bat" ], [ "nat","tan" ], [ "ate","eat","tea" ]]
```

Ejemplo 2:

```
Input: strs = [ " "]
Output: [[ " " ]]
```

Ejemplo 3:

```
Input: strs = [ "a" ]
Output: [[ "a" ]]
```

Usted debe resolver el problema considerando:

- `1 <= strs.length <= 1000`
- `1 <= strs[i].length <= 100`.
- `strs[i]` solo puede contener dígitos y caracteres no especiales (es decir no tiene la ñ, ç, á, etc).

Utilice este prototipo de función para su solución:

```
char*** groupAnagrams(char** strs, int strsSize, int* returnSize, int** returnColumnSizes) {
}
```

5 Ejercicio 4: Estudio lingüístico

Escriba un programa en C que realice un análisis básico de una cadena de caracteres **str** arbitraria. Debe contar la cantidad de vocales, consonantes, dígitos, caracteres especiales (como ñ, ç, á, símbolos, etc.) y palabras presentes en la cadena.

Para ello, dispone de las siguientes funciones:

```
int es_espacio(char c) {
    return c == ' ' || c == '\t' || c == '\n';
}
int es_vocal(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
           c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
}
int es_letra(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}
int es_digito(char c) {
    return c >= '0' && c <= '9';
}
```

Usted debe desarrollar las siguientes funciones:

```
int contar_vocales(char *cadena);
int contar_consonantes(char *cadena);
int contar_digitos(char *cadena);
int contar_caracteres_especiales(char *cadena);
int contar_palabras(char *cadena);
```

Para resolver el problema, considere:

- $1 \leq \text{str.length} \leq 10^4$

Ejemplo:

```
Input: str = "1984 es una novela distópica escrita por el autor británico George Orwell y publicada ↵
en 1949. Es una de las obras literarias más influyentes del siglo XX, conocida por su poderosa ↵
crítica al totalitarismo y su exploración de temas como la manipulación del lenguaje, la ↵
vigilancia masiva y la represión política."
```

```
Output: contar_vocales(str) = 105
        contar_consonantes(str) = 137
        contar_digitos(str) = 8
        contar_caracteres_especiales(str) = 63
        contar_palabras(str) = 51
```

6 Ejercicio 5: Apuestas de Bodoque

Juan Carlos Bodoque le ha solicitado a usted que le ayude a gestionar sus apuestas de caballo. Se le entregará un código para que usted pueda usarlo a su disposición.

```
typedef struct {
    int apuesta, numero_caballo;
    char* nombre_caballo;
} caballos;
```

Su trabajo consiste en crear un array para almacenar sus apuestas; pero lo más importante es que Bodoque está preocupado de que la memoria no dé abasto con sus caballos. Por lo cual, usted deberá implementar este código para crear un arreglo dinámico.

```
typedef struct {
    caballos* struct_caballo;
    size_t cantidad;
    size_t capacidad_array;
} array_dinamico;
```

Usted debe desarrollar las siguientes funciones:

```
void inicializarArray(array_dinamico* arr, size_t capacidad_inicial);
void agregarCaballo(array_dinamico* arr, char* nombre_caballo, int apuesta, int numero_caballo);
void liberarMemoria(array_dinamico* arr);
void imprimirCaballos(array_dinamico* arr);
```

Será un plus si diseña el array dinámico de forma tal que se ajuste a la memoria necesaria por cada caballo extra que se quiera agregar, demostrando su dominio y uso de la memoria en C.

Para casos de prueba, estos son los caballos:

```
Nombre;Apuesta;Numero de Caballo.
Tormenta China; 5000; 1.
Tepo Tepo; 3000; 2.
Coliforme; 200; 3.
Señor Manguera; 500; 4.
Sopapiglobo; 3000; 5.
```

7 REGLAS DE ENTREGA Y CONSIDERACIONES GENERALES

- Este trabajo debe realizarse **individualmente**, vale decir, en **grupos de un (1) estudiante**. No se harán excepciones.
- El programa debe ser desarrollado en lenguaje C, y compilado con la versión de gcc disponible en el servidor Aragorn¹: gcc 4.8.5.
- Cada ejercicio debe ser entregado en archivos separados con su función main respectiva; no hay un límite máximo de funciones a realizar. Si ud. desea modularizar al máximo, tiene la libertad de hacerlo.
- La tarea debe ser entregada en la plataforma **AULA USM**, el día Viernes 13 de Septiembre de 2024 hasta las 23:59:59 Hora de Chile Continental (UTC -4).
- Cada día de atraso implicará un descuento de 20 puntos de la nota final. Se considera como 1 día de atraso luego de las 00:00:00 horas. Ejemplo: Si entrega el sábado 13 de Septiembre a las 00:10:00 horas, se considera como 1 día de atraso. Recomendación: suba su tarea con anticipación.
- La tarea debe incluirse en un archivo comprimido **.tar.gz**. El nombre del archivo debe seguir la siguiente estructura: `tarea1-eda-nombre-apellido-paralelo.tar.gz`; e.g., `tarea1-eda-oliver-atom-p200.tar.gz`.
- El archivo comprimido debe incluir, al menos, lo siguiente:
 - Códigos `.c`: Código del ejercicio solicitado por separado.
 - **README**: Archivo de texto plano en el cual se debe incluir: (1) Sus datos académicos, (2) las condiciones de compilación y ejecución, (3) las instrucciones de compilación y ejecución y (4) casos de prueba con los cuales validó sus ejercicios.
- Cada fuga de memoria será penalizada. Utilice **valgrind** para verificar la correcta asignación, uso y liberación de memoria en la ejecución de su programa.
- Si su ejercicio no compila, su nota será automáticamente **un cero (0)**, será responsabilidad de usted contactar a los ayudantes para una revisión y/o corrección.
- Se realizará un ranking, entre las tareas que hayan sido realizadas de forma completa y correcta. A mayor número de ejercicios completados, mayor probabilidad hay de que el alumno esté en el ranking. Se considerará para el ranking las tres mayores notas de los 5 ejercicios. **Los primeros 5 estudiantes en este ranking, tendrán un bono entre 10 y 15 puntos de acuerdo al lugar obtenido, en cualquier pregunta del certamen 1 de la asignatura.**
- Cualquier atisbo de copia será penalizada con máxima severidad (**nota 0**). Será responsabilidad de usted averiguar con los ayudantes y/o profesor del ramo como resolver este problema.
- No deje la tarea para último momento. Planifique bien sus tiempos y constrúyala paso a paso.
- Consulte sus dudas, lo más pronto posible, a través de los diversos medios de la asignatura.

¹`ssh aragorn.elo.utfsm.cl -l cuentausm`